

Local Web UI Spec

Local AI Software Team

1. Document Overview

Product Name

Local AI Software Team

Document Type

Local Web UI Specification

Purpose

This document defines the local web UI for Local AI Software Team.

The local web UI is a browser-based interface that runs on the user's machine. It is used for settings, workflow control, run history, document review, prompt template editing, logs, reports, traceability, and optional integration setup.

The local web UI is not a cloud SaaS dashboard.

It runs locally at:

```
http://localhost:4317
```

The user starts it with:

```
aiteam ui
```

2. UI Product Summary

The local web UI is the visual control center for the AI software team.

The CLI is still the main engine, but the local web UI makes the product easier to use.

The local web UI helps users:

- configure project settings
- configure AI coding tools
- configure workflow behavior
- input rough requirements

- run workflow stages
- review generated documents
- approve or reject workflow gates
- inspect logs
- view code diff
- view test results
- view final reports
- manage prompt templates
- configure optional integrations

The UI should make the product feel like a real AI software team, not just a text generator.

3. UI Goals

Goal 1: Make setup easy

The user should be able to configure the tool without manually editing JSON files.

Goal 2: Make workflow visible

The user should always know:

- what stage is running
- which agent is responsible
- what has been generated
- what requires approval
- what failed
- what to do next

Goal 3: Make outputs reviewable

Generated artifacts should be easy to read, copy, export, and regenerate.

Goal 4: Make AI actions safe

Risky actions must be clearly visible before the user approves them.

Goal 5: Make local-first obvious

The UI should communicate that the project data, generated docs, logs, and settings are local by default.

4. UI Principles

4.1 Local-first

The UI should make it clear that everything is running locally.

Recommended visual hints:

Local Mode Active
Project data is stored on your machine
No cloud backend connected

4.2 Transparent workflow

Every run should show a workflow timeline.

Example:

Input → Requirement → Scope → Design → Tasks → Tests → Approval → Code →
Review → Report

4.3 Human approval first

The UI should never hide approval gates.

Before code generation, external update, rollback, commit, or PR creation, the UI must show a confirmation screen.

4.4 Document-first

The product should emphasize generated documents and planning outputs before coding.

4.5 Developer-friendly

The UI should be clean, practical, and not overly “consumer app” style.

The user is likely a developer, freelancer, backend engineer, or technical founder.

5. Information Architecture

Main navigation:

Dashboard
New Requirement
Runs
Projects
Settings
Prompt Templates
Integrations

Optional later navigation:

Reports
Traceability
Agent Team Room
Logs
Templates

Recommended MVP sidebar:

Local AI Software Team

- Dashboard
- New Requirement
- Runs
- Settings
- Prompt Templates
- Integrations

6. Global Layout

6.1 App Shell

The local web UI should use a simple app shell.

Left Sidebar

Contains:

- product name
- current project
- navigation items
- local status indicator

Example:

Local AI Software Team

Project:
hotel-booking-service

- Local mode

Dashboard
New Requirement
Runs

Settings
Prompt Templates
Integrations

Top Bar

Contains:

- current page title
- current project selector
- run status if inside a run
- quick actions

Example quick actions:

New Requirement
Start Run
Open Project Folder

Main Content Area

Shows the current page.

Right Panel

Optional contextual panel.

Can show:

- next actions
- approval gates
- agent messages
- run summary
- warnings

7. Global UI States

7.1 Empty State

Used when no project or run exists.

Example:

No project initialized yet.

Start by initializing this repository or selecting a project folder.

```
[Initialize Project]
[Open Existing .ai-team Project]
```

7.2 Loading State

Used when workflow is running.

Example:

```
BA Agent is generating clarified requirement...

This may take a moment.
```

Do not show fake progress percentages unless actual progress is known.

7.3 Error State

Must show:

- what failed
- why it failed if known
- where logs are stored
- suggested next action

Example:

```
Codex CLI was not found.

The selected Developer Agent requires Codex CLI, but it is not installed or
not available in PATH.

Suggested actions:
1. Install Codex CLI.
2. Change Developer Agent in Settings.
3. Run aiteam doctor.
```

7.4 Approval Required State

Used when user action is required.

Example:

```
Approval required before code generation.

The system is about to run Codex CLI in your project folder.
```

Potentially affected areas:

- src/main/java
- src/test/java
- pom.xml

[Approve and Run]

[Edit Plan]

[Cancel]

7.5 Success State

Example:

Workflow completed.

Generated:

- Clarified requirement
- Technical design
- Task breakdown
- Test matrix
- Final report

[View Report]

[Export Artifacts]

[Start New Requirement]

8. Page: Dashboard

8.1 Purpose

The Dashboard gives the user a high-level overview of the current local project and recent workflow runs.

8.2 Main Sections

Current Project Card

Shows:

- project name
- project path
- project type
- framework
- default workflow mode
- default AI agents
- test command status

Example:

```
Project: hotel-booking-service
Path: /Users/hung/projects/hotel-booking-service
Type: Brownfield
Stack: Java / Spring Boot / Maven
Default Mode: Docs-only
Default Developer Agent: Codex
```

Local Status Card

Shows:

- local server running
- config valid
- database accessible
- local mode active
- cloud backend disabled

Example:

```
Local Mode: Active
Backend Cloud: Not used
Project Data: Stored locally
```

AI CLI Status Card

Shows available AI tools.

Example:

```
Claude Code: Available
Codex CLI: Missing
Gemini CLI: Available
Aider: Disabled
```

Each item should have:

- status
- configure button
- test button

Latest Runs

Shows recent workflow runs.

Columns:

- title
- status
- mode
- created date
- last updated
- actions

Example:

Title	Mode	Status	Updated	Action
Export invoice CSV	Docs-only	Report generated	Today	View
Reset password OTP	Semi-auto	Test failed	Today	Resume
Booking cancellation approval	Assisted	Plan generated	Yesterday	View

Quick Actions

Buttons:

[New Requirement]
[Open Latest Run]
[Settings]
[Run Doctor Check]

8.3 Empty State

If no runs exist:

No workflow runs yet.

Start by entering a rough requirement. The AI team will turn it into clear docs, tasks, tests, and reports.

[Create New Requirement]

9. Page: New Requirement

9.1 Purpose

The New Requirement page is where users start a workflow run.

9.2 Main Form Fields

Requirement Input

Large text area.

Placeholder:

Example:
Thêm API export invoice CSV theo hotelId, date range, status.

Requirement Source

Options:

Manual input
Pasted from Jira
Pasted from Slack
Pasted from client message
Pasted from meeting notes

MVP can store this only as metadata.

Input Language

Options:

Auto detect
Vietnamese
English

Default:

Auto detect

Output Language

Options:

English
Vietnamese
Bilingual

Recommended default for Vietnamese developers:

Bilingual

Workflow Mode

Options:

Docs-only
Assisted
Semi-autonomous
Multi-agent

MVP default:

Docs-only

Mode descriptions:

Docs-only:
Generate requirement, design, tasks, test matrix, and final report. No code changes.

Assisted:
Generate implementation prompt for coding agents. User runs the agent manually.

Semi-autonomous:
Run selected AI coding CLI after approval.

Multi-agent:
Coordinate multiple AI roles and tools. Advanced mode.

Target Project

Dropdown:

- current project
- other configured projects
- select folder

Project Type

Options:

Auto detect
Java / Spring Boot
Node.js / NestJS

React / Vite
Generic

Selected Developer Agent

Only shown for assisted or semi-autonomous mode.

Options:

Claude Code
Codex CLI
Gemini CLI
Aider

Max Iterations

Used for semi-autonomous mode.

Default:

3

Output Artifacts

Checkboxes:

Clarified requirement
Acceptance criteria
Technical design
API design
DB design
Task breakdown
Test matrix
Implementation prompt
Final report
Traceability matrix

Default MVP:

Clarified requirement: enabled
Acceptance criteria: enabled
Technical design: enabled
Task breakdown: enabled
Test matrix: enabled
Final report: enabled

9.3 Action Buttons

Primary:

[Start Workflow]

Secondary:

[Save Draft]
[Preview Workflow]
[Clear]

9.4 Preview Workflow

Before starting, user can preview stages.

Example:

This run will execute:

1. BA Agent: clarify requirement
2. PO Agent: define scope
3. Architect Agent: create technical design
4. PM Agent: create task breakdown
5. QA Agent: create test matrix
6. Reporter Agent: create final report

No code will be modified.

9.5 Validation Rules

The form should not start if:

- requirement is empty
- selected project does not exist
- selected mode requires agent but no agent is configured
- semi-auto mode selected but project is not initialized
- output language is missing

9.6 Success Behavior

After starting workflow:

- create new run
- redirect to Run Detail page
- show workflow timeline

- stream status if available

10. Page: Runs

10.1 Purpose

The Runs page lists all workflow runs for the current project.

10.2 Table Columns

Column	Description
Title	Run title
Mode	Docs-only, Assisted, Semi-auto, Multi-agent
Status	Current run status
Created	Created date
Updated	Last updated date
Agent	Selected developer agent if any
Test	Test status if any
Actions	View, Resume, Export, Delete

10.3 Filters

Filters:

Status
Mode
Date range
Agent
Search by title

Status filter options:

Created
Spec generated
Plan generated
Waiting for approval
Code generated
Test passed
Test failed
Review passed
Report generated

Failed
Cancelled

10.4 Actions

Per run:

View
Resume
Export
Duplicate
Delete

Bulk actions:

Export selected
Delete selected
Clean old runs

10.5 Empty State

No runs found.

Create your first workflow run from a rough requirement.

[New Requirement]

11. Page: Run Detail

11.1 Purpose

The Run Detail page is the most important screen.

It shows the full workflow execution, generated artifacts, logs, approvals, test results, diff, and final report.

11.2 Header

Shows:

- run title
- run status
- mode

- created date
- selected project
- selected agent
- actions

Example:

Export invoice CSV

Mode: Docs-only

Status: Report generated

Project: hotel-booking-service

Created: 2026-06-22 14:30

Header actions:

[Resume]

[Regenerate]

[Export]

[Open Folder]

[Delete]

11.3 Workflow Timeline

Horizontal or vertical timeline.

Stages:

Input
Requirement
Scope
Design
Tasks
Tests
Approval
Code
Review
Traceability
Report

Each stage shows:

- status icon
- agent role
- generated artifact count
- error if any

Example statuses:

Not started
Running
Generated
Waiting for approval
Failed
Skipped
Completed

11.4 Run Detail Tabs

Recommended tabs:

Overview
Input
Requirement
Scope
Design
Tasks
Tests
Implementation
Review
Traceability
Report
Logs

MVP tabs:

Overview
Requirement
Design
Tasks
Tests
Report
Logs

12. Run Detail Tab: Overview

12.1 Purpose

Shows summary of the run and next actions.

12.2 Sections

Run Summary

Shows:

- original requirement summary
- current status
- mode
- generated artifacts
- next recommended action

Agent Activity Summary

Example:

```
BA Agent: Completed
PO Agent: Completed
Architect Agent: Completed
PM Agent: Completed
QA Agent: Completed
Reporter Agent: Completed
```

Output Summary

Shows generated outputs:

```
Clarified requirement: Generated
Acceptance criteria: Generated
Technical design: Generated
Task breakdown: Generated
Test matrix: Generated
Final report: Generated
```

Next Action Panel

Examples:

If docs-only completed:

```
Next recommended action:
Review the final report or switch to Assisted Mode to generate an
implementation prompt.
```

Buttons:

[View Final Report]
[Generate Implementation Prompt]
[Export Artifacts]

If waiting for approval:

Approval required before code generation.

Buttons:

[Review Plan]
[Approve Code Generation]
[Reject]

If test failed:

Tests failed. Review the failed tests and decide whether to run fix loop.

Buttons:

[View Test Result]
[Run Fix Loop]
[Generate Report Anyway]

13. Run Detail Tab: Input

13.1 Purpose

Shows original raw requirement and run metadata.

13.2 Content

Fields:

- raw requirement
- source
- input language
- output language
- mode
- created by
- created at

Actions:

[Edit and Rerun]
[Duplicate Run]
[Copy Input]

13.3 Edit and Rerun

If user edits input, the UI should create a new run rather than overwrite the old run.

14. Run Detail Tab: Requirement

14.1 Purpose

Shows requirement clarification outputs.

14.2 Sections

Clarified Requirement

Markdown viewer with edit option.

Business Rules

Table or markdown.

Example table:

ID	Rule	Priority
BR-001	Export must filter by hotelId	High
BR-002	Invalid date range must return error	High

User Roles

Example:

Admin
Hotel Manager
Finance User

Acceptance Criteria

Table or checklist.

Example:

AC-001: User can export invoices by hotelId.
AC-002: User can filter by date range.
AC-003: Invalid date range returns validation error.

Open Questions

Should be highlighted.

Example:

1. What is the maximum export size?
2. Which roles can export invoices?
3. Should CSV include all fields or selected fields?

Assumptions

Example:

A-001: Only authenticated users can export invoices.
A-002: Export file uses UTF-8 encoding.

14.3 Actions

[Approve Requirement]
[Edit]
[Regenerate]
[Answer Questions]
[Continue with Assumptions]
[Copy Markdown]

14.4 Approval Behavior

If requirement approval gate is enabled, the next stages should wait until user approves.

15. Run Detail Tab: Scope

15.1 Purpose

Shows product scope, MVP scope, out-of-scope items, and success criteria.

15.2 Sections

Product Goal

Short summary of user/business value.

MVP Scope

Checklist.

Out of Scope

List of excluded items.

Priority

Table:

Item	Priority	Reason
Export by hotelId	High	Core requirement
Export progress tracking	Low	Not needed in MVP

Success Criteria

Example:

- User can download valid CSV file.
- CSV respects selected filters.
- Invalid filters return clear error.
- Unauthorized user cannot export.

15.3 Actions

```
[Approve Scope]
[Edit]
[Regenerate Smaller Scope]
[Regenerate Detailed Scope]
```

16. Run Detail Tab: Design

16.1 Purpose

Shows technical design artifacts.

16.2 Sections

Repository Analysis

If available:

```
Language: Java
Framework: Spring Boot
Build tool: Maven
Testing: JUnit
Migration: Flyway
```

Technical Design

Markdown viewer.

API Design

For backend features.

Example:

```
GET /api/invoices/export
Query:
- hotelId
- fromDate
- toDate
- status
Response:
- text/csv
```

Database Design

Shows:

- new tables
- modified tables
- migration notes
- indexes
- rollback notes

Service Flow

Can be shown as text or Mermaid diagram.

Example:

Controller → Service → Repository → CSV Generator → Response

Risk Analysis

Examples:

- Large CSV export may cause memory issue.
- Date range validation must be strict.
- Permission check must happen before query.

16.3 Actions

[Approve Design]
[Edit]
[Regenerate]
[Make Simpler]
[Make More Detailed]
[Copy Markdown]

17. Run Detail Tab: Tasks

17.1 Purpose

Shows task breakdown and Jira-ready format.

17.2 Sections

Epic

Example:

Epic: Invoice CSV Export

Stories

Cards or table.

Each story shows:

- title
- description
- acceptance criteria

- priority
- estimate
- dependencies

Subtasks

Nested under story.

Jira-ready Markdown

Copyable block for Jira.

17.3 Task Table

Type	Title	Priority	Estimate	Status
Epic	Invoice CSV Export	High	-	Ready
Story	Add export API	High	M	Ready
Subtask	Implement controller endpoint	High	S	Ready

17.4 Actions

[Copy Jira Markdown]
 [Export JSON]
 [Regenerate Tasks]
 [Split Tasks Smaller]
 [Merge Tasks]
 [Create Jira Tasks] later

18. Run Detail Tab: Tests

18.1 Purpose

Shows test matrix, manual checklist, and test execution results.

18.2 Sections

Test Matrix

Table:

ID	Requirement	Scenario	Type	Expected Result	Priority
TC-001	AC-001	Export by hotelId	Integration	CSV returned	High

ID	Requirement	Scenario	Type	Expected Result	Priority
TC-002	AC-003	Invalid date range	Unit	Validation error	High

Manual Test Checklist

Checklist UI.

Example:

```
[ ] Login as authorized user
[ ] Open invoice export endpoint
[ ] Export with valid filters
[ ] Verify CSV headers
[ ] Verify CSV rows
```

Test Execution Result

Only available after `aiteam test`.

Shows:

- command
- exit code
- duration
- status
- failed tests
- logs

18.3 Actions

```
[Regenerate Test Matrix]
[Run Tests]
[View Test Logs]
[Copy Test Matrix]
[Export Test Matrix]
```

19. Run Detail Tab: Implementation

19.1 Purpose

Shows implementation prompt, coding checklist, agent output, changed files, and diff.

This tab is relevant for Assisted, Semi-autonomous, and Multi-agent modes.

19.2 Sections

Implementation Prompt

Markdown viewer.

Actions:

```
[Copy Prompt]
[Edit Prompt]
[Regenerate Prompt]
```

Coding Checklist

Example:

- Follow existing project conventions.
- Do not modify unrelated files.
- Add unit tests.
- Add validation.
- Do not edit .env or production config.

Selected Agent

Shows selected AI CLI.

Example:

```
Developer Agent: Codex CLI
Status: Available
```

Agent Output

Log viewer.

Changed Files

Table:

File	Status	Risk
InvoiceExportController.java	Modified	Normal
InvoiceExportService.java	Modified	Normal
pom.xml	Modified	Warning

Diff Viewer

Shows patch/diff.

Should support:

- file-by-file view
- collapse files
- copy diff
- open file locally

19.3 Actions

```
[Approve Code Generation]
[Run Agent]
[Stop Agent]
[Copy Prompt]
[View Diff]
[Rollback Changes]
[Continue to Test]
```

19.4 Risk Warnings

If protected or warning files changed:

```
Warning: High-risk file changed.

pom.xml was modified.
Review carefully before continuing.
```

If protected file changed:

```
Protected file modified.

.env was modified. Workflow is stopped until you review the change.
```

20. Run Detail Tab: Review

20.1 Purpose

Shows code review, security review, and requirement coverage review.

20.2 Sections

Review Summary

Example:

Status: Changes required

Requirement Coverage

Table:

Requirement	Status	Notes
Export by hotelId	Covered	Implemented in service
Invalid date range validation	Partial	Test missing

Code Quality Review

Shows:

- maintainability issues
- complexity issues
- convention issues
- unrelated changes

Security Review

Shows:

- auth/authz issues
- input validation issues
- secret leakage
- sensitive logging
- risky file changes

Required Fixes

Checklist:

- ☐ Add test for empty CSV result
- ☐ Add date range validation
- ☐ Verify permission check

20.3 Actions

```
[Run Review]
[Run Security Review]
[Run Fix Loop]
[Mark as Accepted]
[Generate Report Anyway]
```

21. Run Detail Tab: Traceability

21.1 Purpose

Shows mapping from requirement to tasks, code, and tests.

21.2 Before Code

If code has not been generated, show:

```
Requirement → Acceptance Criteria → Task → Test Case
```

21.3 After Code

If code has been generated, show:

```
Requirement → Acceptance Criteria → Task → Code File → Test Case → Test
Result
```

21.4 Table

Requirement	AC	Task	Code	Test	Status
Export by hotelId	AC-001	TASK-001	InvoiceExportService.java	TC-001	Covered
Reject invalid date range	AC-003	TASK-003	InvoiceFilterValidator.java	TC-002	Partial

21.5 Status Types

```
Covered
Partial
Not covered
Unknown
```

21.6 Actions

[Regenerate Traceability]
[Export Markdown]
[Export JSON]
[Copy Table]

22. Run Detail Tab: Report

22.1 Purpose

Shows final delivery report.

22.2 Report Sections

The final report should include:

- original requirement
- clarified requirement
- assumptions
- open questions
- scope summary
- technical summary
- task summary
- test summary
- implementation summary
- changed files
- review summary
- traceability summary
- risks
- next steps

22.3 Actions

[Regenerate Report]
[Copy Markdown]
[Export]
[Open in Folder]
[Post to Slack] later
[Comment to Jira] later

23. Run Detail Tab: Logs

23.1 Purpose

Shows workflow logs, shell logs, AI agent logs, test logs, and integration logs.

23.2 Log Types

Tabs inside Logs:

```
Workflow
Agent
Shell
Test
Integration
Error
```

23.3 Log Viewer Features

Should support:

- search
- copy
- download
- auto-scroll
- filter by level

Log levels:

```
Info
Warning
Error
Debug
```

23.4 Secret Masking

Logs should mask secret-like values.

Example:

```
Authorization: Bearer sk-*****
```

24. Page: Settings

24.1 Purpose

The Settings page lets users configure local project behavior without editing config files.

24.2 Settings Sections

```
Project
AI CLI
Agent Mapping
Workflow
Test Commands
Safety
Documentation
Storage
```

25. Settings Section: Project

25.1 Fields

```
Project Name
Project Path
Project Type
Main Language
Framework
Build Tool
Default Output Language
```

25.2 Example

```
Project Name: hotel-booking-service
Project Path: /Users/hung/projects/hotel-booking-service
Project Type: Brownfield
Language: Java
Framework: Spring Boot
Build Tool: Maven
Default Output Language: Bilingual
```

25.3 Actions

```
[Save]
[Auto-detect Project]
[Open Project Folder]
```

26. Settings Section: AI CLI

26.1 Purpose

Configure available AI coding tools.

26.2 Fields Per Tool

For each AI CLI:

```
Enabled
Command path
Availability status
Default timeout
Test command button
```

Tools:

```
Claude Code
Codex CLI
Gemini CLI
Aider
```

26.3 Status Values

```
Available
Missing
Disabled
Error
```

26.4 Example UI

```
Claude Code
Status: Available
Command: claude
```

[Disable] [Test]

Codex CLI
Status: Missing
Command: codex
[Enable] [Test]

Gemini CLI
Status: Available
Command: gemini
[Disable] [Test]

27. Settings Section: Agent Mapping

27.1 Purpose

Map AI roles to AI tools.

27.2 Fields

BA Agent
Product Owner Agent
Project Manager Agent
Architect Agent
Developer Agent
QA Agent
Reviewer Agent
Security Reviewer Agent
Reporter Agent

Each field is a dropdown:

Claude Code
Codex CLI
Gemini CLI
Aider
Default
Disabled

27.3 Recommended Defaults

BA Agent: Gemini
Product Owner Agent: Gemini
Project Manager Agent: Gemini

```
Architect Agent: Claude
Developer Agent: Codex
QA Agent: Codex
Reviewer Agent: Claude
Security Reviewer Agent: Claude
Reporter Agent: Gemini
```

If a tool is not available, show warning.

28. Settings Section: Workflow

28.1 Fields

```
Default Workflow Mode
Require Requirement Approval
Require Plan Approval
Require Code Approval
Max Fix Iterations
Default Input Language
Default Output Language
Generate Traceability by Default
```

28.2 Defaults

```
Default Workflow Mode: Docs-only
Require Requirement Approval: false
Require Plan Approval: true
Require Code Approval: true
Max Fix Iterations: 3
Default Input Language: Auto
Default Output Language: Bilingual
Generate Traceability: true
```

29. Settings Section: Test Commands

29.1 Purpose

Configure build/test commands.

29.2 Fields

```
Build Command
Unit Test Command
Integration Test Command
Lint Command
Type Check Command
Custom Command
```

29.3 Java/Spring Boot Example

```
Build Command: mvn clean package -DskipTests
Unit Test Command: mvn test
Integration Test Command: mvn verify
Lint Command:
```

29.4 Node Example

```
Build Command: npm run build
Unit Test Command: npm test
Lint Command: npm run lint
Type Check Command: npm run typecheck
```

29.5 Actions

```
[Save]
[Test Commands]
[Auto-detect]
```

30. Settings Section: Safety

30.1 Purpose

Configure safety rules.

30.2 Fields

```
Require approval before code generation
Require approval before commit
Require approval before external update
Require approval before rollback
```

```
Command timeout
Max iterations
Protected files
Warning files
Blocked commands
Secret masking enabled
```

30.3 Default Protected Files

```
.env
.env.*
*.pem
*.key
credentials.json
application-prod.yml
application-production.yml
```

30.4 Default Warning Files

```
pom.xml
build.gradle
package.json
Dockerfile
.github/workflows/*
```

30.5 Default Blocked Commands

```
sudo
rm -rf /
chmod 777
curl | sh
wget | sh
mkfs
dd if=
```

30.6 UI Warning

Show:

```
Safety rules protect your local project from risky AI actions.
Do not disable these unless you understand the risk.
```

31. Settings Section: Documentation

31.1 Purpose

Configure generated documentation outputs.

31.2 Fields

Checkboxes:

Generate clarified requirement
Generate PRD-style summary
Generate SRS-style requirement
Generate technical design
Generate API design
Generate DB design
Generate task breakdown
Generate test matrix
Generate implementation prompt
Generate review report
Generate traceability matrix
Generate final report

31.3 Output Format

Options:

Markdown
JSON
HTML later
PDF later

MVP default:

Markdown
JSON for structured outputs

32. Settings Section: Storage

32.1 Purpose

Show where local data is stored.

32.2 Fields

```
.ai-team folder path  
Database path  
Runs folder path  
Prompt templates path  
Logs folder path
```

32.3 Actions

```
[Open .ai-team Folder]  
[Open Runs Folder]  
[Clean Old Runs]  
[Export Settings]  
[Import Settings]
```

33. Page: Prompt Templates

33.1 Purpose

Allows users to view and edit agent prompt templates.

33.2 Template List

Templates:

```
BA Agent  
Product Owner Agent  
Project Manager Agent  
Architect Agent  
Developer Agent  
QA Agent  
Reviewer Agent  
Security Reviewer Agent  
Reporter Agent
```

33.3 Template Editor

Features:

- markdown editor
- variable autocomplete
- preview mode
- reset to default

- save
- duplicate template

33.4 Supported Variables

Examples:

```
{{rawRequirement}}  
{{clarifiedRequirement}}  
{{acceptanceCriteria}}  
{{businessRules}}  
{{technicalDesign}}  
{{taskBreakdown}}  
{{testMatrix}}  
{{repositoryAnalysis}}  
{{diff}}  
{{testResult}}
```

33.5 Validation

The UI should warn if template references unknown variables.

Example:

```
Unknown variable: {{technicalPlanSummary}}
```

33.6 Actions

```
[Save]  
[Preview]  
[Reset to Default]  
[Duplicate]  
[Validate]
```

34. Page: Integrations

34.1 Purpose

Configure optional external integrations.

The product must work without integrations.

34.2 Integration Cards

Cards:

```
GitHub
Jira
Slack
GitLab later
Confluence later
Notion later
Microsoft Teams later
```

Each card shows:

- enabled/disabled
- connection status
- required credentials
- test connection button
- last check result

35. Integration: GitHub

35.1 Modes

```
GitHub CLI mode
Personal access token mode
```

Recommended MVP:

```
GitHub CLI mode
```

35.2 Fields

```
Enabled
Mode
Owner
Repository
Default base branch
Use gh CLI
```

35.3 Actions

```
[Test Connection]
[Create PR from Run] later
[Read CI Result] later
```

35.4 Warning

Creating PRs requires approval. The product will not push or create PRs automatically without confirmation.

36. Integration: Jira

36.1 Fields

```
Enabled
Jira Site URL
Email
Project Key
Default Issue Type
API Token
```

Token should be stored securely.

36.2 MVP Behavior

MVP can support:

```
Generate Jira-ready Markdown
```

Later:

```
Create Epic
Create Story
Create Subtask
Comment Final Report
Update Status
```

36.3 Actions

```
[Test Connection]
[Generate Jira-ready Tasks]
[Create Jira Tasks] later
```

36.4 Warning

Jira updates require approval. The product will not create or update issues automatically without confirmation.

37. Integration: Slack

37.1 Fields

```
Enabled
Bot Token
Default Channel ID
Post Progress Updates
Post Final Report
```

37.2 MVP Behavior

Post-only.

Supported later:

```
Post progress update
Post final report
Post PR-ready notification
```

Not MVP:

```
Receive Slack commands
Listen to Slack events
```

37.3 Actions

[Test Connection]
[Send Test Message]

37.4 Warning

Receiving Slack events requires a public callback URL and is not supported in local-only MVP.

38. Page: Projects

38.1 Purpose

Manage multiple local projects.

This page can be MVP+ if initial product supports only current folder.

38.2 Project List

Columns:

Project	Path	Type	Last Run	Status
---------	------	------	----------	--------

38.3 Actions

[Add Project]
[Open Project]
[Remove from List]
[Run Doctor]
[Open Folder]

38.4 Add Project Form

Fields:

Project Name
Project Path
Project Type
Default Output Language

Actions:

```
[Select Folder]
[Auto-detect]
[Add]
```

39. Page: Agent Team Room

39.1 Purpose

The Agent Team Room makes the workflow feel like a real AI software team.

This can be MVP+.

39.2 Layout

Chat-like timeline of agent messages.

Agents:

```
BA Agent
PO Agent
PM Agent
Architect Agent
Developer Agent
QA Agent
Reviewer Agent
Security Reviewer Agent
Reporter Agent
```

39.3 Example Messages

```
BA Agent:
I found 3 missing business rules: export permission, maximum CSV size, and
empty result behavior.

Architect Agent:
The proposed endpoint is GET /api/invoices/export. It affects
InvoiceController, InvoiceService, and InvoiceRepository.

QA Agent:
I created 8 test cases. 5 normal cases, 2 edge cases, and 1 permission case.
```

Reviewer Agent:
Review found one issue: missing validation for fromDate greater than toDate.

39.4 Actions

User can:

Reply to agent
Ask agent to revise
Approve agent output
Reject agent output
Pin message
Copy message

39.5 Message Types

Info
Question
Decision
Warning
Error
Approval Request
Result

40. Approval UI

40.1 Purpose

Approval UI ensures user control.

40.2 Approval Types

Requirement approval
Scope approval
Design approval
Plan approval
Code generation approval
Risky file approval
External update approval
Rollback approval

40.3 Approval Modal Content

Must show:

- action to approve
- reason approval is needed
- affected files or external tools
- risk level
- generated summary
- available alternatives

40.4 Example: Code Generation Approval

Approval required before code generation.

The system will run Codex CLI in:

/Users/hung/projects/hotel-booking-service

The agent may modify files under:

- src/main/java
- src/test/java

Protected files:

- .env
- application-prod.yml

Selected agent:

Codex CLI

[Approve and Run]

[Cancel]

[Edit Implementation Prompt]

40.5 Example: External Update Approval

Approval required before posting to Slack.

Channel:

#dev-updates

Message:

Final report for Export Invoice CSV is ready.

[Post to Slack]

[Cancel]

[Edit Message]

41. Notifications and Toasts

41.1 Success Toasts

Examples:

```
Requirement generated.  
Technical design generated.  
Test matrix generated.  
Final report generated.  
Settings saved.
```

41.2 Warning Toasts

Examples:

```
Codex CLI is not available.  
Protected file change detected.  
Jira token is missing.  
Some acceptance criteria are not covered.
```

41.3 Error Toasts

Examples:

```
Workflow failed.  
Test command failed.  
Could not read config file.  
Could not start local server.
```

Each error toast should include a link to logs or details when possible.

42. Search and Navigation

42.1 Global Search

Future feature.

Search across:

- runs
- generated docs
- reports

- tasks
- prompts

42.2 Run Search

MVP should support search by:

- run title
 - requirement text
 - status
 - date
-

43. Export and Copy Behavior

43.1 Copy Buttons

Every generated artifact should have:

Copy Markdown
Copy Plain Text

43.2 Export Options

Supported:

Markdown folder
ZIP
JSON

Future:

HTML
PDF
Confluence
Notion

43.3 Export Modal

Fields:

Run
Format
Include logs

```
Include diff
Include JSON
Output folder
```

44. Accessibility Requirements

The UI should support:

- keyboard navigation
 - visible focus state
 - readable contrast
 - semantic headings
 - button labels
 - error messages near fields
 - scrollable code/log areas
 - large text areas for requirements and prompts
-

45. Responsive Behavior

The local web UI should work best on desktop browser.

Minimum supported width:

```
1024px
```

At smaller widths:

- sidebar can collapse
- run detail tabs can become dropdown
- tables can scroll horizontally

Mobile is not a priority for MVP.

46. Visual Style

46.1 Tone

Professional, developer-focused, clean.

Avoid overly playful design.

46.2 Recommended Visual Style

- light/dark mode later

- simple sidebar
- card-based dashboard
- markdown-first document viewer
- clear status badges
- clear warning colors
- readable monospace blocks for logs/diff

46.3 Status Badges

Examples:

```
Generated
Running
Waiting for approval
Failed
Skipped
Completed
Warning
```

47. MVP UI Scope

47.1 Must-have Pages

```
Dashboard
New Requirement
Runs
Run Detail
Settings
Prompt Templates
```

47.2 Must-have Features

```
Start docs-only workflow
View workflow status
View generated docs
View run history
Edit basic settings
Edit prompt templates
View logs
Export markdown
```

47.3 Should-have Features

- Integration settings
- AI CLI status check
- Test command settings
- Traceability view
- Approval modal
- Agent Team Room basic view

47.4 Could-have Features

- Diff viewer
- Test result viewer
- GitHub/Jira/Slack action buttons
- Dark mode
- Project manager
- Global search

47.5 Out of Scope for MVP

- Cloud account login
- Team collaboration
- Real-time Slack event receiving
- Full Jira automation
- PR creation
- Auto deployment
- Mobile app
- Desktop app
- Billing UI
- Enterprise admin console

48. Example MVP User Journey

48.1 First-time Setup

1. User runs:

```
aiteam ui
```

1. Browser opens:

`http://localhost:4317`

1. UI shows setup screen.
2. User sets:

Project path
Project type
Default output language
AI CLI availability
Test command
Safety settings

1. User clicks:

Save Settings

48.2 First Requirement Run

1. User opens New Requirement.
2. User enters:

`Thêm API export invoice CSV theo hotelId, date range, status.`

1. User selects:

Mode: Docs-only
Output language: Bilingual

1. User clicks Start Workflow.
2. UI redirects to Run Detail.
3. Timeline shows each agent stage.
4. User opens Requirement tab.
5. User reviews acceptance criteria.
6. User opens Design tab.
7. User reviews API design.
8. User opens Tasks tab.
9. User copies Jira-ready tasks.
10. User opens Tests tab.
11. User reviews test matrix.
12. User opens Report tab.
13. User exports markdown.

48.3 Expected Result

User gets a complete delivery package without code modification.

49. Future UI Roadmap

Phase 1: Local Docs UI

- Dashboard
- New Requirement
- Runs
- Run Detail
- Settings
- Prompt Templates
- Docs viewer
- Logs viewer
- Markdown export

Phase 2: Assisted Coding UI

- Implementation prompt viewer
- Copy prompt
- Coding checklist
- Expected changed files

Phase 3: Semi-auto Coding UI

- Approval screen
- AI CLI status
- Agent logs
- Changed files
- Diff viewer
- Test result viewer
- Review report

Phase 4: Traceability UI

- Traceability table
- Coverage status
- Requirement-to-test mapping
- Requirement-to-code mapping

Phase 5: Integration UI

- GitHub PR summary
- Jira task creation

Slack post final report
Confluence/Notion publishing

50. Final UI Summary

The local web UI should be the control center for the local AI software team.

It should help the user move from:

rough requirement

to:

clear requirement
technical design
task breakdown
test matrix
implementation prompt
review report
traceability
final report

The UI must stay local-first, safe, transparent, and reviewable.

The most important MVP screens are:

Dashboard
New Requirement
Run Detail
Settings
Prompt Templates
Runs

The Run Detail screen is the heart of the product.

The product should feel like a structured AI team workflow, not a generic AI chat interface.